

ERP Website Order Management System

Development Document

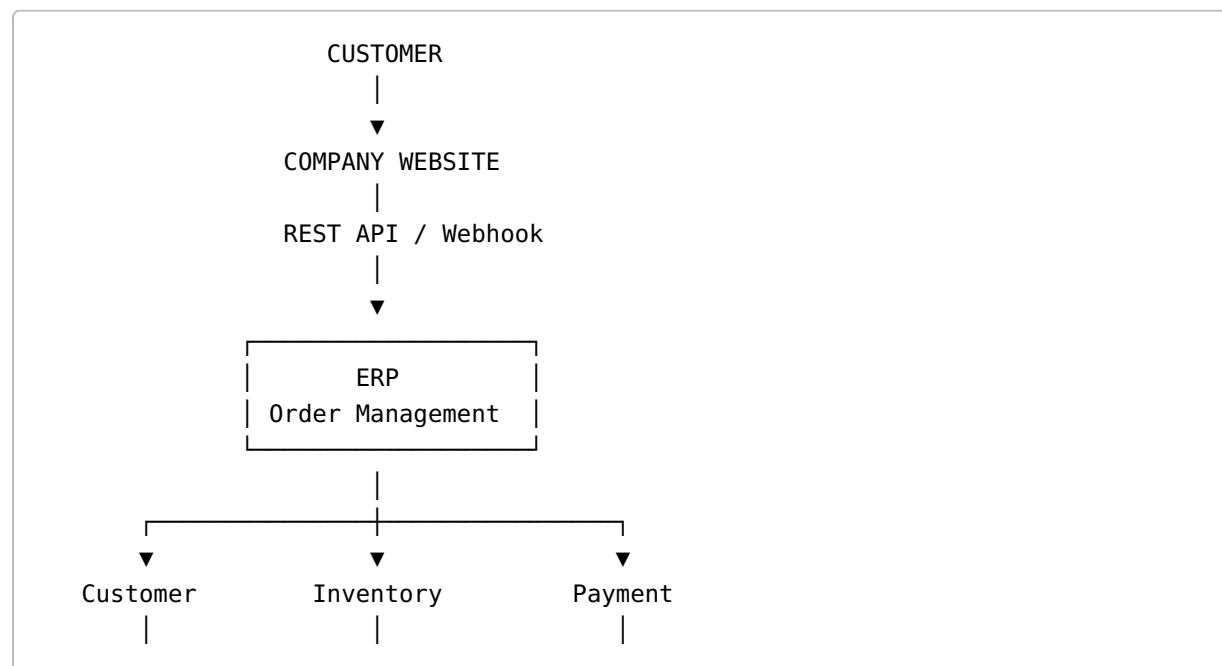
1. Project Objective

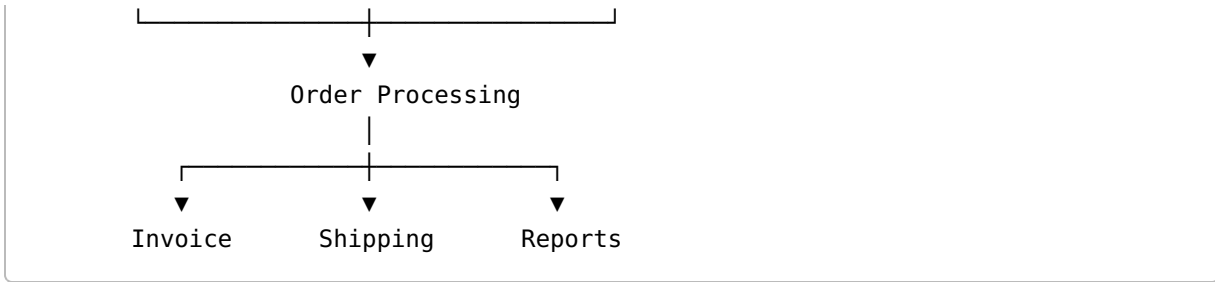
Develop an ERP-based order management system that connects with the company's website and automatically fetches, manages, processes, and synchronizes website orders.

The ERP will act as the central system for:

- Website orders
- Customers
- Products
- Inventory
- Payments
- Invoices
- Shipping
- Returns/refunds
- Order status
- Reports
- Staff operations

2. System Architecture





3. Main ERP Modules

3.1 Authentication & Users

The system should support:

- Login
- Logout
- Password reset
- Role-based access
- User permissions
- Session management
- Login activity
- Audit logs

Roles

- Super Admin
- Admin
- Manager
- Sales Staff
- Inventory Staff
- Accounts Staff
- Shipping Staff
- Viewer

4. Website Order Fetcher

This is the main module.

The ERP should fetch orders from the website automatically.

Order data

Order ID
Website Order ID
Customer
Products
Quantity
Price
Discount
Tax
Shipping Charge
Grand Total
Payment Status
Payment Method
Shipping Address
Billing Address
Order Date
Order Status

Order statuses

NEW
↓
CONFIRMED
↓
PROCESSING
↓
PACKED
↓
SHIPPED
↓
DELIVERED

Alternative flows:

NEW → CANCELLED

PROCESSING → CANCELLED

DELIVERED → RETURN REQUEST
↓
RETURNED

↓
REFUNDED

5. Order Synchronization

The ERP should synchronize website and ERP data.

Website → ERP

Fetch:

- New orders
- Customer details
- Product information
- Payment status
- Shipping information
- Order updates
- Cancellation
- Return requests

ERP → Website

Send:

- Order status
- Stock availability
- Product stock
- Invoice information
- Shipping/tracking information
- Cancellation status
- Refund status

6. API/Webhook System

The website should communicate with the ERP through APIs.

Example endpoints

```
POST /api/orders
GET  /api/orders
GET  /api/orders/{id}
PUT  /api/orders/{id}
```

```
POST /api/orders/{id}/confirm
POST /api/orders/{id}/cancel
POST /api/orders/{id}/ship
POST /api/orders/{id}/deliver
POST /api/orders/{id}/return
POST /api/orders/{id}/refund
```

Webhooks

The website can notify the ERP when:

```
order.created
order.updated
order.cancelled
payment.completed
payment.failed
shipment.created
shipment.updated
order.delivered
return.created
refund.completed
```

7. Customer Management

The ERP should automatically create or update customers when an order is received.

Customer information

```
Customer ID
Name
Phone
Email
Billing Address
Shipping Address
Total Orders
Total Purchases
Last Order
Customer Status
```

Duplicate prevention

Before creating a customer:

```
Check Phone
  ↓
Check Email
  ↓
Existing Customer?
├─ YES → Update Customer
└─ NO  → Create Customer
```

8. Product Management

Products should be centrally managed.

Product fields

```
Product ID
SKU
Product Name
Category
Brand
Description
Selling Price
Purchase Price
Tax
Stock
Minimum Stock
Status
Website Status
```

Product synchronization

```
ERP Product
├─ Price → Website
├─ Stock → Website
├─ Status → Website
└─ Product Data → Website
```

9. Inventory Management

Inventory should automatically change based on orders.

Stock flow

```
graph TD;
    A[Available Stock] --> B[Order Created];
    B --> C[Reserve Stock];
    C --> D[Order Confirmed];
    D --> E[Deduct Stock];
    E --> F[Order Shipped];
```

The system should maintain:

- Available stock
- Reserved stock
- Sold stock
- Returned stock
- Damaged stock
- Warehouse stock
- Low-stock alerts

10. Warehouse Management

Support multiple warehouses.

```
graph TD;
    subgraph Warehouse_A [Warehouse A]
        direction TB
        A1[Product A]
        A2[Product B]
        A3[Product C]
    end
    subgraph Warehouse_B [Warehouse B]
        direction TB
        B1[Product A]
        B2[Product D]
        B3[Product E]
    end
```

Order allocation:

```
Website Order
  ↓
Check Stock
  ↓
Nearest/Configured Warehouse
  ↓
Reserve Stock
  ↓
Packing
```

11. Payment Management

The ERP should track payment information.

Payment methods

- Online Payment
- Cash on Delivery
- Bank Transfer
- UPI
- Card
- Wallet
- Other Gateway

Payment statuses

```
PENDING
PAID
FAILED
PARTIAL
REFUNDED
PARTIALLY_REFUNDED
```

Payment records should include:

```
Transaction ID
Order ID
Payment Gateway
Amount
Payment Method
```

Payment Status
Payment Date
Refund Amount

12. Invoice Management

Invoices should be generated from confirmed orders.

Invoice information

Invoice Number
Order Number
Customer
Products
Quantity
Price
Discount
Tax
Shipping
Grand Total
Payment Status
Invoice Date

Functions:

- Generate invoice
- Download PDF
- Print invoice
- Email invoice
- Credit note
- Refund invoice

13. Shipping Management

Shipping staff should manage the delivery process.

Shipping fields

Order ID
Courier

Tracking Number
Shipping Date
Expected Delivery
Current Status
Delivered Date

Shipping statuses

READY_TO_SHIP
PACKED
SHIPPED
IN_TRANSIT
OUT_FOR_DELIVERY
DELIVERED
FAILED
RETURN_TO_ORIGIN

14. Return & Refund Management

Customers can request returns.

Flow

Delivered
↓
Return Request
↓
Admin Review
↓
Approved / Rejected
↓
Product Received
↓
Quality Check
↓
Refund

Refund should be linked to:

- Original order
- Original payment
- Refund amount

- Refund date
- Refund transaction ID
- Refund status

15. Order Dashboard

Dashboard should display:

Today's Orders
Today's Sales
Pending Orders
Processing Orders
Shipped Orders
Delivered Orders
Cancelled Orders
Return Requests
Pending Payments
Low Stock Products

Example:

ERP DASHBOARD		
Orders 125	Sales ₹85,500	Pending 23
Shipped 32	Delivered 58	Cancelled 7

16. Order Management Screen

The order list should support:

- Search
- Filter
- Sort
- Date filter
- Customer filter

- Payment filter
- Status filter
- Warehouse filter
- Export

Order table

Order ID
Customer
Items
Amount
Payment
Status
Warehouse
Created Date
Actions

17. Notifications

The system should support:

Internal notifications

- New order
- Payment received
- Low stock
- Order cancellation
- Return request
- Refund request
- Failed payment

Customer notifications

- Order confirmation
- Payment confirmation
- Order packed
- Order shipped
- Delivery update
- Order delivered
- Return confirmation
- Refund confirmation

Channels can include:

- Email

- WhatsApp
 - SMS
 - ERP notification
-

18. Reports

Required reports:

Sales

- Daily sales
- Weekly sales
- Monthly sales
- Product-wise sales
- Customer-wise sales
- Category-wise sales

Orders

- New orders
- Completed orders
- Cancelled orders
- Returned orders
- Pending orders

Inventory

- Current stock
- Low stock
- Stock movement
- Warehouse stock
- Product movement

Payments

- Paid
- Pending
- Failed
- Refunds

Shipping

- Shipped
- Delivered
- Delayed
- Failed delivery

- Returned shipments

Reports should support:

```
Filter
Search
Date Range
Excel Export
CSV Export
PDF Export
```

19. Audit Log

Every important operation should be recorded.

Example:

```
User
Action
Module
Record ID
Old Value
New Value
IP Address
Timestamp
```

Examples:

```
Admin changed order status
Manager changed product price
Staff cancelled order
Accounts processed refund
Inventory staff adjusted stock
```

20. API Security

All APIs must implement:

- Authentication

- Authorization
- Role-based permissions
- API rate limiting
- Input validation
- Request logging
- Secure tokens
- HTTPS
- Webhook signature verification
- Replay protection
- Audit logging

Sensitive information must not be exposed through API responses.

21. Order Fetcher Reliability

The order fetcher must be designed to prevent duplicate orders.

Idempotency

```
Website Order ID
  ↓
Check ERP
  ↓
Already Exists?
├─ YES → Update existing order
└─ NO  → Create new order
```

Each external order should have a unique identifier.

Failed synchronization

```
Fetch Order
  ↓
Success?
├─ YES → Save Order
└─ NO
    ↓
  Retry Queue
    ↓
  Retry
    ↓
Success / Failed
```

The system should record synchronization failures instead of silently losing orders.

22. Background Jobs

Heavy operations should run through queues.

Examples:

```
Order Synchronization
Inventory Synchronization
Invoice Generation
Email Sending
WhatsApp Notifications
Shipping Updates
Report Generation
Webhook Processing
```

This prevents slow API requests from blocking the ERP.

23. Database Core Tables

Recommended tables:

```
users
roles
permissions

customers
customer_addresses

products
product_categories
product_variants

warehouses
warehouse_stocks
stock_movements

orders
order_items
order_status_history
```

```
payments
payment_transactions
refunds

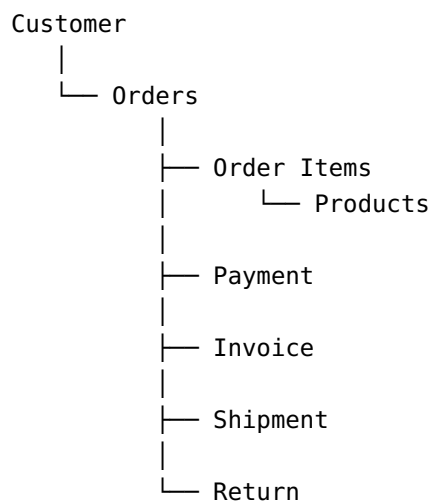
invoices
invoice_items

shipments
shipment_tracking

returns
return_items

notifications
webhooks
sync_logs
audit_logs
```

24. Important Database Relationships



25. Development Phases

Phase 1 — Foundation

- Project setup

- Database
- Authentication
- Roles
- Permissions
- Admin dashboard

Phase 2 — Master Data

- Customers
- Products
- Categories
- Warehouses
- Inventory

Phase 3 — Website Integration

- API authentication
- Order API
- Webhooks
- Order fetcher
- Sync engine
- Duplicate prevention

Phase 4 — Order Processing

- Order management
- Payment
- Invoice
- Packing
- Shipping
- Delivery

Phase 5 — Returns

- Return requests
- Approval
- Product receiving
- Refund

Phase 6 — Reports

- Sales reports
- Order reports
- Inventory reports
- Payment reports
- Shipping reports

Phase 7 — Security & Testing

- API security
- Permission testing
- Validation testing
- Load testing
- Integration testing
- Backup testing
- Audit testing

Phase 8 — Production

- Server deployment
- HTTPS
- Database backup
- Queue workers
- Cron jobs
- Monitoring
- Error logging
- Production testing

26. Recommended Technology

Backend

```
Laravel  
REST API  
PostgreSQL / MySQL  
Redis  
Laravel Queue  
Laravel Scheduler
```

Frontend

```
React  
Vite  
Tailwind CSS
```

Infrastructure

```
Linux VPS  
Nginx
```

PHP - FPM
Redis
Database Server
SSL
Cloudflare

27. Development Principle

The ERP should **not depend on manual order entry**.

The primary workflow should be:

```
WEBSITE
↓
AUTOMATIC ORDER FETCH
↓
VALIDATE
↓
CREATE / UPDATE CUSTOMER
↓
CREATE ORDER
↓
RESERVE INVENTORY
↓
VERIFY PAYMENT
↓
GENERATE INVOICE
↓
PACK
↓
SHIP
↓
DELIVER
↓
UPDATE WEBSITE
↓
REPORT
```

The system should be designed as an **API-first, event-driven order management platform**, allowing additional websites, marketplaces, payment gateways, and shipping providers to be integrated later without rebuilding the core ERP.